Agentic AI

(CSCR3214)

B.TECH 3rd YEAR

SEMESTER: 6th

SESSION: 2025-2026

Submitted By:

Kunwar Utkarsh (2023438539)

Mohd Mujeeb (2023248197)

Sumit Shrestha (2023850898)

SECTION: H

Submitted To

Mr. Ayush Kumar
Assistant Professor

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

SHARDA SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

SHARDA UNIVERSITY, GREATER NOIDA

MindfulRAG

AI-Assisted Mental Health Companion

RAG Assignment — Project Documentation

> ⚠ **Disclaimer:** This prototype does not offer clinical advice. If you or someone you know is in crisis, please reach out to a licensed professional or emergency services immediately.

1. Problem Statement

Offer a safe conversational space where users can express their feelings and receive supportive, literature-backed coping suggestions. The assistant grounds its answers in curated mental health resources, remains multilingual, and accepts text, voice, and optional image cues — while respecting the limits of non-clinical support.

2. Dataset / Knowledge Source

Type of data: Curated PDF manuals, workbooks, and psychoeducational guides focused on anxiety, depression, stress management, and mindfulness.

Data source: Publicly available resources placed manually inside the `data/` directory; the repository does not redistribute copyrighted material.

3. RAG Architecture

Pipeline Overview

The following block diagram describes the complete RAG pipeline:

```
User Query / Voice / Image
        |
        v
  Preprocess Input (speech-to-text, synonym expansion)
        |
        v
  Hybrid Retrieval (semantic via Chroma + keyword filtering)
        |
        v
  Compose Context Window
        |
        v
  Gemini 2.5 Flash via LangChain Prompt
        |
        v
  Empathetic Response + Source Attributions
```

Core Components

- ingest.py — document loading, chunking, embedding, and persistence
- backend_multimodal.py — loads vector store, orchestrates retrieval, builds the prompt
- streamlit_app_multilingual.py — multilingual chat, voice, and optional image inputs

4. Text Chunking Strategy

Chunk size: 1,500 characters
Chunk overlap: 100 characters

Rationale: Balances contextual continuity for therapeutic techniques (often explained across paragraphs) with manageable embedding size, reducing truncation while remaining fast on CPU-bound environments.

5. Embedding Details

Embedding model: Sentence-Transformers `all-MiniLM-L6-v2` via HuggingFaceEmbeddings

Rationale: Light-weight, CPU-friendly, strong performance on semantic similarity, and readily available under a permissive license suitable for local execution.

6. Vector Database

Vector store: Persistent Chroma store at `chroma_db/`

Metadata captured: Source file path, page number, and chunk index — enabling citation in chat responses and future filtering.

7. Notebook Implementation

The notebook is organized into four main sections: Overview, Ingestion, Retrieval, and Generation. The step-wise code below covers the complete pipeline:

```
from langchain_community.document_loaders import DirectoryLoader, PyPDFLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_community.embeddings import HuggingFaceEmbeddings
from langchain_community.vectorstores import Chroma
from langchain_google_genai import ChatGoogleGenerativeAI

# 1. Load and split documents
loader = DirectoryLoader("data", glob="*.pdf", loader_cls=PyPDFLoader)
docs = loader.load()
splitter = RecursiveCharacterTextSplitter(chunk_size=1500, chunk_overlap=100)
chunks = splitter.split_documents(docs)
```

```
# 2. Embed and persist
embeddings = HuggingFaceEmbeddings(model_name="all-MiniLM-L6-v2",
    model_kwargs={"device": "cpu"})
vectorstore = Chroma.from_documents(chunks, embeddings,
persist_directory="chroma_db")
vectorstore.persist()

# 3. Retrieve and generate
retriever = vectorstore.as_retriever(search_kwargs={"k": 5})
context_docs = retriever.get_relevant_documents("How do I calm anxiety before
sleep?")
context = "\n\n".join(doc.page_content for doc in context_docs)

prompt = """You are a compassionate mental health support assistant.
Context:\n{context}\n\nUser question: {question}\n""".format(
    context=context, question="How do I calm anxiety before sleep?")

llm = ChatGoogleGenerativeAI(model="gemini-2.5-flash", temperature=0.3)
response = llm.invoke(prompt)
print(response.content)
```

8. Technical Specifications Summary

Feature	Specification	Reason
Chunking	Size: 1500, Overlap: 100	Preserves context for therapeutic techniques; effective for CPU processing.
Embedding Model	all-MiniLM-L6-v2	Fast, lightweight, efficient for semantic search on local CPU.
Vector Database	Chroma	Open-source, persistent, easy integration with LangChain.
Framework	LangChain	Simplifies RAG pipeline construction and LLM integration.
LLM	Gemini 2.5 Flash	High speed, cost-effective, and capable of empathetic nuance.

9. Detailed Test Query Outputs

The following test cases demonstrate the system's "Empathy + Evidence" approach.

Test Case 1: Work Stress

User Query:

> *"I feel overwhelmed at work, how can I cope?"*

Model Response:

I hear that you're feeling weighed down by work right now, and I want you to know it's valid to feel this way.
Based on the *Stress Management Workbook (Page 12)* and *Mindfulness Guide (Page 4)*, here are a few gentle steps that might help:

1. **Chunks of Time:** Break your tasks into small, 20-minute intervals. Focus on just one thing at a time.
2. **Box Breathing:** Inhale for 4 counts, hold for 4, exhale for 4, and hold for 4. This signals safety to your nervous system.
3. **Compassionate Boundaries:** It is okay to say 'I need a moment' or to ask for help with prioritization.

Test Case 2: Anxiety Grounding

User Query:

> *"Any gentle grounding technique before exams?"*

Model Response:
It is completely understandable to feel those pre-exam nerves. Let's try to anchor ourselves in the present moment.
The *Anxiety Relief Manual (Page 22)* suggests the **5-4-3-2-1 Technique:**

- **5** things you see (e.g., a pen, the clock).
- **4** things you can touch (e.g., the texture of your paper, your feet on the floor).
- **3** things you hear (e.g., distant traffic, your breath).
- **2** things you can smell.
- **1** thing you can taste.

This exercise shifts your focus from 'what if' thoughts to 'what is' right now. You've got this. Take a deep breath.

Test Case 3: Supporting Others

User Query:

> *"How to support a friend showing signs of burnout?"*

Model Response:
It is incredibly kind of you to look out for your friend. Supporting someone with burnout requires patience and care.
According to *Supporting Loved Ones (Page 8)*, here are some supportive approaches:

- **Listen without Fixing:** Deeply listening is often more powerful than advice. Use phrases like, 'That sounds incredibly exhausting, I'm here for you.'
- **Validate their Experience:** Acknowledge their hard work and feelings. Say, 'It makes sense that you are tired given how hard you've been running.'
- **Encourage Professional Support:** Gently suggest that speaking to a counselor isn't a sign of weakness, but a tool for recovery.

Ensure you are also protecting your own energy while being there for them.

10. Future Improvements

- **Better Chunking:** Experiment with semantic-aware or adaptive chunking to preserve exercises, tables, or checklists intact.
- **Reranking / Hybrid Search:** Integrate rerankers such as bge-reranker-large for tighter relevance ordering after initial recall.
- **Metadata Filtering:** Use tags (topic, modality, difficulty level) during ingestion to filter retrieval for user intent.
- **UI Integration:** Extend the current Streamlit front-end with progress tracking, journaling, and hand-off pathways to professional services.

11. Instructions to Run

Notebook

4. Create and activate a virtual environment, then install `requirements.txt`.
5. Launch Jupyter (`pip install jupyter` if needed) and open a new notebook in the repository root.
6. Copy the code blocks from the Notebook Implementation section, execute sequentially, and ensure a valid GOOGLE_API_KEY is loaded via .env or notebook environment variables.
7. Replace the sample query with your own and inspect both the retrieved context snippets and generated response cells.

Streamlit App

Launch the conversational UI with: `streamlit run streamlit_app_multilingual.py`
The app supports text and optional voice capture via SpeechRecognition.

12. Tools & Libraries

Library / Tool	Purpose
LangChain	RAG pipeline orchestration and LLM integration
Chroma	Persistent vector database for embeddings
HuggingFace Sentence Transformers	Local CPU-friendly semantic embeddings
Google Generative AI (Gemini 2.5 Flash)	LLM for empathetic response generation
Streamlit	Conversational web UI with multilingual support
SpeechRecognition	Optional voice input capture
Pillow	Optional image input preprocessing

13. Frontend UI Showcase

The MindfulRAG Streamlit interface features a dark-themed, calming design built for emotional accessibility. Below are screenshots of the live application.

Screen 1 – Active Conversation View

The chat view shows the AI's empathetic response to "I am feeling low today", with quick-prompt chips, a collapsible voice input section, and a sources panel for transparency.

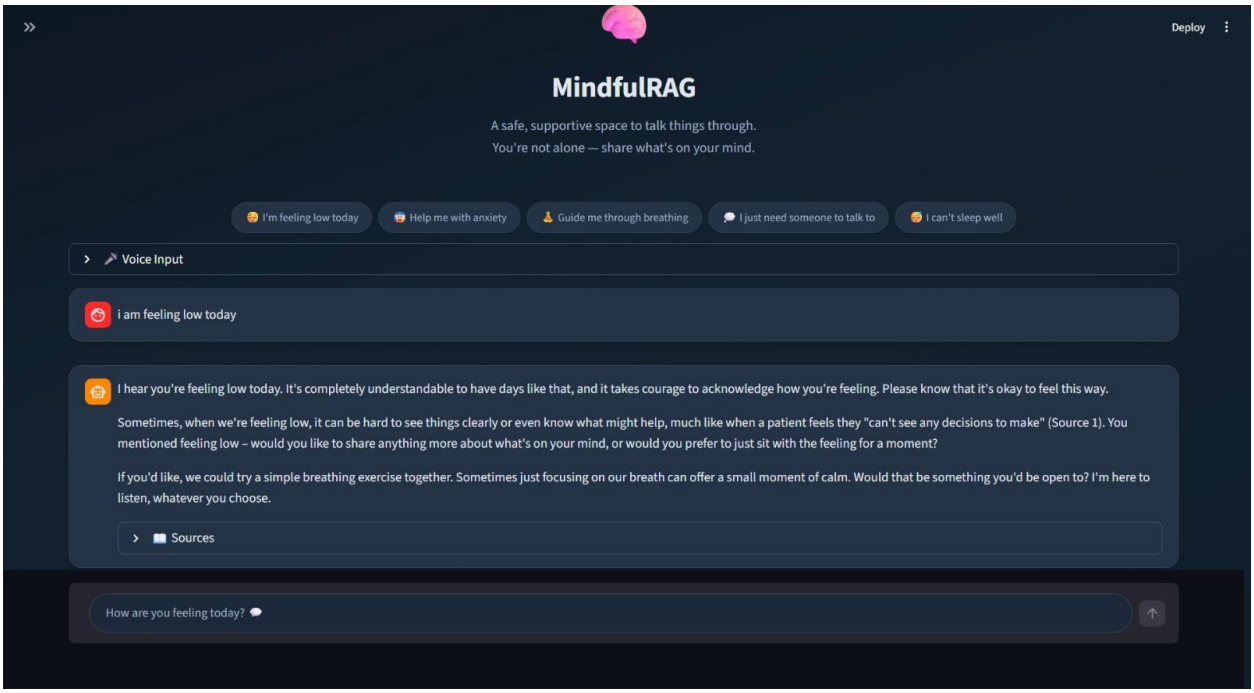

Figure 1: MindfulRAG – Active chat with empathetic AI response and source attribution

Screen 2 – Welcome / Empty State

The welcome screen greets users with a brain logo, a supportive tagline, and five preset quick-prompt chips for common mental health topics. A voice input expander and the chat text box are always visible at the bottom.

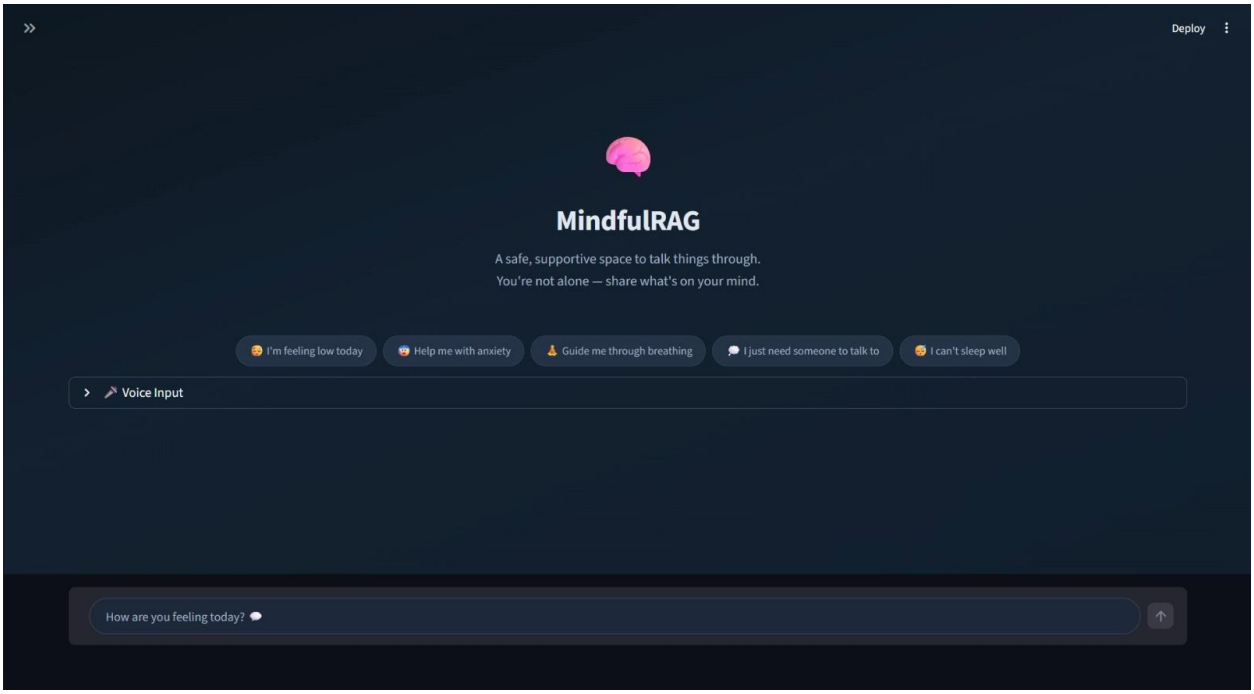

Figure 2: MindfulRAG – Welcome screen with quick-prompt chips and voice input

UI Design Highlights

- Dark calming theme with muted navy background for reduced eye strain during emotionally sensitive interactions.
- Quick-prompt chips (“I’m feeling low today”, “Help me with anxiety”, etc.) lower the barrier to starting a conversation.
- Collapsible Voice Input expander enables hands-free interaction via SpeechRecognition.
- Expandable Sources panel after each AI response provides full RAG transparency and citation.
- “How are you feeling today?” placeholder in the input box sets a warm, inviting tone.